Task 1 – Web App QA & Debug Report

Application Tested

RealWorld (Conduit) Demo Application

Objective

The objective of this QA assessment was to evaluate the application's core user workflows and identify functional, usability, accessibility, performance, and security issues that could negatively impact production usage.

User Flows Tested

- User Registration (Sign Up)
- User Login
- Create Article
- Edit Article
- Delete Article
- Commenting
- Profile Management
- Logout

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Cross-Site Scripting (XSS) via Markdown Content	Login → Create article → Insert `<script>alert('XSS')</script>` or `[Click Me](javascript:alert('XSS'))` → Publish → Open article	**Expected:** User content is sanitized and scripts removed. **Actual:** JavaScript executes in browser.	Critical	Missing sanitization during markdown-to-HTML rendering; unsafe content rendered directly.
2	Missing Character Limit	Go to Settings → Paste bio >10,000 characters →	**Expected:** Validation message and character limit	Medium	No client-side validation and insufficient

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
	Validation on User Bio	Click Update Settings	enforcement. **Actual:** Request submitted and may trigger server error.		server-side input constraints.
3	No Loading Indicator During Login/Registration	Open Login/Sign Up → Simulate slow network → Submit form	**Expected:** Loading spinner, disabled submit button, prevention of duplicate requests. **Actual:** No feedback; users can click multiple times.	Medium	Loading state not implemented in authentication workflow.
4	Accessibility Issues (ARIA Labels & Color Contrast)	Navigate application with screen reader and keyboard only → Inspect pagination/navigation controls	**Expected:** Proper ARIA labels, keyboard accessibility, WCAG-compliant contrast. **Actual:** Missing labels and low-contrast text in some areas.	Low	Accessibility standards not considered during UI implementation.
5	Comments Section Loads All Comments at Once	Open article containing hundreds of comments	**Expected:** Pagination, lazy loading, or virtualized rendering. **Actual:** All comments load simultaneously causing slow performance.	Medium	No pagination or lazy-loading strategy implemented for comments.

Root Cause Analysis Example (BUG #1)

Issue: Cross-Site Scripting (XSS) via Markdown Content

The application stores article content exactly as entered by users and renders it after converting markdown into HTML. During rendering, malicious HTML tags and JavaScript payloads are not properly sanitized. Because the browser treats the generated HTML as trusted content, scripts can execute in the user's session. This creates a security vulnerability that could lead to session hijacking, data theft, or account compromise. The likely root cause is the absence of a sanitization layer between user input, markdown

processing, and browser rendering. To fix the issue, both frontend and backend validation should be implemented. A library such as DOMPurify should sanitize HTML before rendering, and the backend should reject dangerous payloads before storing content. Additionally, a Content Security Policy (CSP) should be configured as a defense-in-depth measure.